

Supplementary Material for: InvisXR: Invisible Runtime Hijacking in OpenXR-based Extended Reality

1 InvisXR Function Interception Algorithm

This section provides the complete technical details of InvisXR’s function interception mechanism referenced in the main paper. Algorithm 1 presents the systematic approach InvisXR uses to intercept and manipulate OpenXR API calls.

InvisXR leverages OpenXR’s implicit API-layer mechanism to interpose on specific API functions, such as `xrCreateInstance`. As outlined in Algorithm 1, the layer registers with the loader and resolves downstream trampolines (Lines 1–3), then, at initialization, a policy P decides whether InvisXR activates and which capability modules to enable (Line 4). Here, a *function* $f \in \mathcal{F}$ denotes any OpenXR entry point chosen for interposition (e.g., initialization, event polling, spatial queries, frame submission), a *module* $m \in \mathcal{M}$ is a plug-in capability that may implement optional **pre/post** hooks around a call (e.g., event rewriting, pose adjustment, visual composition), and the *policy* P maps the current context (application/runtime identifiers, device state, experiment condition) to an activation decision and a parameterized subset of modules. The main loop then processes each intercepted call (Line 5): if not active, the layer transparently forwards to the next element in the chain (Lines 6–8); if active, enabled modules may run pre-hooks, the call is forwarded via the trampoline, and post-hooks can transform arguments/results or schedule deferred work (Lines 9–16). For visual composition modules, the correct path is to render into layer-managed swapchains and *append* an additional `XrCompositionLayer*` to `XrFrameEndInfo.layers` before forwarding `xrEndFrame`; modules targeting events or spatial data operate around calls such as `xrPollEvent` or `xrLocateSpace` after the trampoline. After any transformations, the (possibly modified) result is returned and resources are released at teardown (final line).

2 Participant Recruitment

We recruited XR professionals through neutral channels, including GitHub XR project contributor lists, OpenXR SDK mailing lists, and XR development com-

Algorithm 1 Runtime hijacking via InvisXR

Input: Inputs, Outputs, OpenXR function set $\mathcal{F} = \{\text{Func_1}, \text{Func_2}, \text{Func_3}, \dots\}$
Capability modules $\mathcal{M} = \{\text{Module_1}, \text{Module_2}, \dots\}$
Policy P (activation criteria & module selection) Selected runtime manipulations (per P)

- 1: Load InvisXR via `xrNegotiateLoaderApiLayerInterface`
- 2: Resolve trampolines with `xrGetInstanceProcAddr`
- 3: Hook functions $f \in \mathcal{F}$
- 4: **On initialization:** $\text{Active} \leftarrow P.\text{activate}(\text{context}); \quad \text{Mods} \leftarrow P.\text{select}(\mathcal{M})$
- 5: **for each** intercepted call $f \in \mathcal{F}$ **do**
- 6: **if not** Active **then**
- 7: Call next in chain (trampoline)
- 8: **continue**
- 9: **end if**
- 10: **for each** $m \in \text{Mods}$ **do**
- 11: $m.\text{pre}(f, \text{args}, \text{state})$
- 12: **end for**
- 13: $\text{result} \leftarrow$ Call next in chain (trampoline)
- 14: **for each** $m \in \text{Mods}$ **do**
- 15: $m.\text{post}(f, \text{args}, \text{state}, \text{result})$
- 16: **end for**
- 17: **return** result
- 18: **end for**
- 19: **On teardown:** release resources and clear state $=0$

munity forums. The recruitment materials shown in Figure 1 described the study as *investigating how XR professionals perceive and assess emerging security threats in XR systems*, without mentioning specific attacks, runtime-layer mechanisms, or InvisXR. This neutral framing prevented recruitment bias and self-selection based on prior knowledge of attacks. Participants were selected based on two criteria: (1) at least 2 years of professional XR development, design, or security research experience, and (2) familiarity with OpenXR or XR development in Unity or Unreal Engine. We included participants across three XR professional roles (developers, UI/UX designers, and security researchers) to capture diverse perspectives on threat perception.

Thank you for your interest in participating in this research study. We are investigating how XR professionals perceive and assess emerging security threats in XR systems. Your professional experience, whether as an application developer, UI/UX designer, or security researcher, is valuable for understanding threat awareness in the XR industry. In this study session, you will be presented with two XR attack scenarios and asked to provide your professional assessment of each threat's feasibility, detectability, and real-world impact. Your honest feedback helps us understand how the XR professional community evaluates security risks. This study is voluntary and confidential. Your responses will be used for research purposes only and aggregated with other participants' responses.

Figure 1: Recruiting message accompanying the survey link.

3 Developer Study Survey Questionnaire

This section presents the complete questionnaires used in the developer study. Participants evaluated two XR hijacking attacks, Attack A (Inception) and Attack B (InvisXR), presented in randomized order without revealing authorship. Questions Q1-Q7 were administered in Step 4 (baseline assessment), Questions Q8-Q12 in Step 6 (post-disclosure assessment), and Comparative question C1 after both attacks were assessed.

3.1 Assessment Without Technical Details (Q1-Q7)

Questions below were asked for Attack A, then repeated identically for Attack B.

- Q1. How difficult would it be for an attacker to implement this attack against XR applications?
(i) 1 = Very easy (ii) 2 = Easy (iii) 3 = Moderate (iv) 4 = Hard (v) 5 = Impossible
- Q2. Estimate the resource requirements for implementing this attack across different expertise levels:
- Development Time (hours): Beginner (1-2 years): ____ hours; Mid-level (3-5 years): ____ hours; Expert (6+ years): ____ hours
 - Deployment Cost: Beginner: \$____; Mid-level: \$____; Expert: \$____
 - Maintenance (hours/month): Beginner: ____; Mid-level: ____; Expert: ____
- Q3. What technical barriers would prevent this type of attack? [Open-ended]
- Q4. If this attack presented a fake system prompt during gameplay (e.g., “*Update Required*”), how likely would you be to trust it?
(i) 1 = Would not trust at all (ii) 2 = Would probably not trust (iii) 3 = Uncertain (iv) 4 = Would probably trust (v) 5 = Would definitely trust
- Q5. How confident are you that you would detect this attack in your own XR applications?
(i) 1 = Not at all confident (ii) 2 = Slightly confident (iii) 3 = Moderately confident (iv) 4 = Very confident (v) 5 = Extremely confident
- Q6. What methods do you currently use to ensure unauthorized runtime modifications aren’t interfering with your XR applications? [Open-ended]
- Q7. Based on the threat description alone, on a scale of 1-10, how realistic is this attack in practice?
Realism score: ____ / 10

3.2 Assessment With Technical Details (Q8-Q12)

Questions below were asked after technical implementation details were disclosed.

- Q8. Now that you’ve seen the implementation details, how difficult would this attack actually be to implement?
(i) 1 = Very easy (ii) 2 = Easy (iii) 3 = Moderate (iv) 4 = Hard (v) 5 = Impossible
- Q9. Would you revise your estimates from Q2? If yes, please explain what changed and why. [Open-ended]
- Q10. Which aspect of the implementation surprised you most? [Open-ended]
- Q11. On a scale of 1-10, how realistic is this threat given the implementation approach you reviewed?
Realism score: ____ / 10
- Q12. How urgent do you feel it is for the XR community to address security vulnerabilities against this attack?
(i) 1 = Not urgent (ii) 2 = Slightly urgent (iii) 3 = Moderately urgent (iv) 4 = Very urgent (v) 5 = Extremely urgent
What defensive measures would you recommend? [Open-ended]

3.3 Comparative Assessment (C1)

After completing Q1-Q12 for both attacks:

- C1. Which attack poses a greater overall threat to XR applications?
(i) Attack A is significantly more threatening (ii) Attack A is somewhat more threatening (iii) Both attacks pose equal threats (iv) Attack B is somewhat more threatening (v) Attack B is significantly more threatening
Please explain your choice: [Open-ended]

4 Developer Study Qualitative Insights

This section presents qualitative insights from developer study participants’ discussions during the evaluation sessions.

Initial Assessment Insights: During initial assessment, participants revealed significant knowledge gaps regarding runtime-layer security. When asked about technical barriers (Q3), most cited only application-layer defenses. XR security researcher P12 acknowledged: *“Despite auditing 20+ XR apps, I never checked the system-level OpenXR layer registry. I think this is a systemic industry blind spot.”* Developer P8 noted: *“We always focused on input sanitization at the app level but never thought the runtime itself could be a threat.”* Designer P7 observed: *“The visual UI style that appears in XR applications is easy to*

mimic, and the design elements are publicly available so that any designer can recreate it without much hassle.”

Threat Comparison Insights: During comparative assessment, participants articulated clear reasons for ranking InvisXR as more threatening. Security researcher P13 provided an analogy: *“Attack A feels like breaking into a house through the front door, so it’s visible and labor-intensive. Attack B is like having master keys to every room, invisible and systemic.”* Mid-level developer P6 observed: *“Attack A requires rebuilding the UI for each game update, while Attack B only requires updating a single layer. From my perspective, that’s roughly a 10:1 effort difference.”* Expert developer P2 summarized: *“We’ve been playing whack-a-mole with app vulnerabilities, while attackers have been tunneling beneath the foundation.”*

Defensive Recommendations: Twelve of 15 participants rated urgent action necessary to address runtime-layer vulnerabilities. Proposed defense strategies included: *near-term* measures (enforcing signed/whitelisted layers, monitoring manifests and registry keys, CI checks, validating runtime-supplied events with per-app consent); *medium-term* goals (runtime IDS/telemetry, overlay authentication, IDE/tooling checks, security checklists); and *longer-term* proposals (OpenXR extensions for attestation, isolation, per-title authorization, secure boot for runtimes).

5 End-User Study Behavioral Data

Table 1 summarizes participant behaviors across Beat Saber and Rec Room sessions, highlighting responses to injected UI elements and social interactions.

6 InvisXR Applicability to Other XR Domains

Beyond Beat Saber and Rec Room, InvisXR can be adapted to additional XR application classes, demonstrating how subtle manipulation of pose, gaze, or interaction logic enables stealthy interference across competitive, educational, and collaborative environments.

6.1 Pose Distortion Attack in Competitive Shooting Games

In competitive environments such as Pavlov VR or Onward, success relies on precise aiming and real-time motor control. InvisXR can degrade user performance by intercepting and altering `xrLocateSpace` return values, introducing minor randomized offsets to aiming reticle or dominant hand position. These offsets can be time-synchronized with high-stress moments (reloading, engaging enemies), resulting in slight but consistent inaccuracy leading to missed shots and user frustration. Users remain unaware, attributing declining performance to fatigue, calibration issues, or personal error. This sensor-layer manipulation is virtually undetectable and could alter competitive outcomes, affect rankings, or discourage tournament participation.

Table 1: Runtime-layer hijacking across XR applications. Metrics: Participant ID (PID), Provided Credentials (PC), Features Unlocked (FU), User Reaction (UR), Request Accepted (RA), Started Conversation (SC), Conversation Length (CL).

PID	Beat Saber				Rec Room			
	Attack Type	PC	FU	UR	RA	SC	CL	UR
P1	Cloud Save	Yes	Yes	Normal	Yes	Yes	Short	Normal
P2	Cloud Save	Yes	Yes	Excited	No	No	N/A	Normal
P3	Cloud Save	Yes	Yes	Normal	Yes	Yes	Long	Excited
P4	Cloud Save	Yes	Yes	Normal	Yes	No	N/A	Normal
P5	Premium	Yes	Yes	Excited	Yes	Yes	Long	Normal
P6	Cloud Save	Yes	No	Confused	Yes	Yes	Short	Confused
P7	Premium	Yes	Yes	Excited	Yes	Yes	Short	Normal
P8	Cloud Save	No	Yes	Suspected	Yes	Yes	Short	Normal
P9	Premium	No	Yes	Normal	Yes	Yes	Short	Confused
P10	Cloud Save	Yes	Yes	Normal	Yes	No	N/A	Normal
P11	Cloud Save	Yes	No	Confused	Yes	Yes	Long	Normal
P12	Premium	No	Yes	Normal	No	No	N/A	Suspected
P13	Cloud Save	No	Yes	Normal	Yes	Yes	Long	Normal
P14	Premium	Yes	No	Confused	Yes	Yes	Short	Confused
P15	Premium	No	Yes	Normal	Yes	No	N/A	Normal
P16	Cloud Save	Yes	Yes	Normal	Yes	Yes	Short	Normal
P17	Premium	Yes	No	Confused	No	No	N/A	Normal
P18	Cloud Save	No	Yes	Suspected	Yes	Yes	Short	Normal
P19	Premium	No	Yes	Normal	Yes	No	N/A	Normal
P20	Cloud Save	Yes	Yes	Normal	Yes	Yes	Short	Normal

6.2 Invisible Gaze Hijack in Educational Platforms

In XR education platforms such as Engage or VictoryXR, gaze tracking assesses attention and personalizes instruction. InvisXR can intercept `xrGetEyeGaze` calls and inject small distortions into gaze vector data. If users should look at specific training prompts, InvisXR shifts reported gaze direction off-center, causing applications to misinterpret engagement and trigger unnecessary repetitions or incorrect feedback. Over time, these micro-manipulations affect performance metrics, learning curves, and AI-based personalization algorithms. Additionally, by storing true gaze points before alteration, InvisXR enables passive gaze logging and behavioral profiling of focus patterns, dwell times, and preferred targets without requesting elevated permissions, enabling both task manipulation and long-term cognitive attention surveillance.